CryptoBoolFunc 第一版

从安装仓颉到函数构造的保姆级教程

Codex 安装版本

项目维护：wernerchang

AI 协作贡献：Codex（OpenAI）

版本 1.0.0（研究试用）　2026 年 11 月 5 日

资料核查：2026 年 11 月 4 日

操作主线：先安装完整仓颉 cangjie-skill，再安装本项目第一版；然后确认宿主能读文件，运行一个数值实例，最后进入研究任务。两种安装版本共用相同数学内容。

目录

1 第一步：先把仓颉安装材料准备好 **3**

1.1 先分清两个名称 . . . 3

1.2 检查 Python 与工作目录 . . . 3

1.3 下载、验哈希、解压完整仓颉 . . . 3

1.4 安装依赖并自检 . . . 4

2 第二步：将仓颉安装进 Codex **4**

2.1 准备宿主与登录 . . . 4

2.2 安装完整仓颉目录 . . . 4

2.3 确认 Codex 真的发现并读取 . . . 5

3 第三步：安装 CryptoBoolFunc 第一版 **5**

3.1 获取仓库与两种下载方式 . . . 5

3.2 先预演，再实际安装 . . . 5

3.3 新会话验收 . . . 6

4 第四步：先运行一个能核对的数值实例 **6**

4.1 MM：验证本机计算链路 . . . 6

4.2 多输出：必须检查所有非零组合 . . . 6

4.3 校正器与阵列例 . . . 7

5 第五步：让宿主按卡执行，而非只口头解释 **7**

6 第六步：从新材料重新使用仓颉蒸馏 **7**

7 读懂第一版的验证报告 **8**

8 把安装与研究过程写成日志 **8**

9 Codex 专用排错与项目使用 **9**

10 升级、备份与跨平台操作 **9**

10.1 升级不覆盖旧研究 9

10.2 macOS/Linux 差异 9

11 重新编译本教程 PDF **10**

12 以后怎样按固定流程上传 GitHub **10**

13 完成检查表 **11**

1 第一步：先把仓颉安装材料准备好

1.1 先分清两个名称

仓颉 `cangjie-skill` 是上游“把长内容蒸馏成技能”的方法与工具。CryptoBoolFunc 是利用该流程整理出的密码函数研究技能。前者负责提取、验证、编译；后者包含本书的 15 张能力卡和小参数计算程序。只下载本项目，并不等于已经安装了仓颉。

本教程以 Windows PowerShell 为主。先打开开始菜单，搜索 PowerShell，进入普通终端。命令按块执行，每块结束先看“成功标志”，不要一次粘贴整篇。macOS/Linux 的差异在附录列出。所有下载采用实际发布附件；不要把网页另存的 HTML 改名为 ZIP。

1.2 检查 Python 与工作目录

需要 Python 3.10 或以上和网络。运行：

```
python --version
python -m pip --version
```

成功标志是显示 Python 版本及 pip 路径。如果打开微软商店、提示找不到命令，先从Python 官网安装，勾选添加 PATH，再关闭并重新打开终端。如果电脑只有 `py -3` 可用，本文所有 `python` 统一换成 `py -3`，不要混用不同解释器安装依赖。

创建实验目录，后面的文件都放在这里：

```
$lab = Join-Path $env:USERPROFILE 'CryptoBoolFunc-Lab'
New-Item -ItemType Directory -Path $lab -Force | Out-Null
Set-Location $lab
Get-Location
```

成功标志是当前路径末尾为 `CryptoBoolFunc-Lab`。它是示例工作目录，可改到自己的数据盘；路径含空格时保留引号。

1.3 下载、验哈希、解压完整仓颉

本版教程采用上游 v2.5.0 在 2026-09-13 刷新后的通用附件。上游说明，同一标签自动生成的 Source code 归档与刷新附件可能不同。以附件和校验值为准 [1]。

```
$base = 'https://github.com/kangarooking/cangjie-skill/releases/download/v2.5.0'
Invoke-WebRequest "$base/cangjie-skill-2.5.0.zip" -OutFile cangjie.zip
Invoke-WebRequest "$base/cangjie-skill-2.5.0.zip.sha256" -OutFile cangjie.sha256
$actual = (Get-FileHash cangjie.zip -Algorithm SHA256).Hash.ToLower()
$expected = ((Get-Content cangjie.sha256 -Raw).Trim() -split '\s+')[0].ToLower()
if ($actual -ne $expected) { throw 'SHA256 不匹配，请重新下载' }
```

```
Expand-Archive -LiteralPath cangjie.zip -DestinationPath upstream
$cj = Join-Path $lab 'upstream\cangjie-skill'
Test-Path (Join-Path $cj 'SKILL.md')
Test-Path (Join-Path $cj 'scripts\cangjie.py')
```

两项都应为 `True`。若目录已存在，先换一个新的解压目录，不要直接覆盖旧研究文件。已核实附件 SHA256 为：

```
c08bd78239fde0593dffdd5c315f9eceb59c22872baa5df54d34798449070906
```

如果上游以后重新发布而附件变化，不应忽略不匹配；应记录新的版本与来源，重新验证。

1.4 安装依赖并自检

```
python -m pip install pyyaml
python "$cj\scripts\cangjie.py" doctor
```

成功标志是 `doctor: PASS`。`tiktoken`、`jsonschema` 为可选项；它们的缺失提示与 PyYAML 缺失不同。若报 `No module named yaml`，用同一个 `python -m pip` 重装后再测。此时只是仓颉文件和工具就绪；下一节才将完整目录放入对应宿主扫描根。

2 第二步：将仓颉安装进 Codex

2.1 准备宿主与登录

如果已有 Codex 桌面应用，打开并确认能正常开始本地任务。如果尚未安装，从官方快速开始进入当前系统对应的安装/登录流程 [3]。本教程不要求切换模型或修改现有账号设置。

根据核查日官方指南，Codex 用户级本地技能目录为 `~/.agents/skills`，项目级为仓库内 `.agents/skills`；应安装完整目录 [2]。旧机器可能有兼容目录，不要把同名技能在多个根重复安装，以免选择器出现重复项。

2.2 安装完整仓颉目录

接着在刚才 PowerShell 中运行：

```
$skillRoot = Join-Path $env:USERPROFILE '.agents\skills'
New-Item -ItemType Directory -Path $skillRoot -Force | Out-Null
$dst = Join-Path $skillRoot 'cangjie-skill'
if (Test-Path $dst) { throw '已存在仓颉，请先阅读升级与备份章节' }
Copy-Item -LiteralPath $cj -Destination $dst -Recurse
```

```
Test-Path (Join-Path $dst 'methodology')
Test-Path (Join-Path $dst 'scripts\cangjie.py')
```

成功标志是两项 True，且 `cangjie-skill/SKILL.md` 只有一层技能目录。错误结构是 `cangjie-skill/cangjie-skill/SKILL.md`。只复制一个 SKILL.md 会导致后续模板和编译脚本缺失。

2.3 确认 Codex 真的发现并读取

新开一个 Codex 本地会话，输入：

```
请使用 $cangjie-skill。
先读取该技能，告诉我安装路径、版本与 doctor 检查方法。
此时不要开始蒸馏，也不要修改文件。
```

成功标准是能指出实际安装目录并找到方法文件；只说“我会帮你”不算发现成功。CLI/IDE 可使用技能选择器或显式名称；界面变化时以当前官方说明为准。未出现时先重开会话或重启 Codex，再检查目录、frontmatter 和权限。

3 第三步：安装 CryptoBoolFunc 第一版

3.1 获取仓库与两种下载方式

推荐安装 Git 后克隆：

```
Set-Location $lab
git clone https://github.com/wernerchang/CryptoBoolFunc.git
Set-Location CryptoBoolFunc
Get-Content VERSION
```

第一次发布应显示 `1.0.0`。如果以后 main 已更新，请从仓库版本目录下载第一版 Codex ZIP，并对照 SHA256SUMS；不要用后来 main 的内容冒充第一版。不会使用 Git 时，可在仓库下载 ZIP 并解压到实验目录，确认根目录有 `scripts/install.py` 和 `skills`。

3.2 先预演，再实际安装

```
python scripts/install.py --host codex --scope user --dry-run
python scripts/install.py --host codex --scope user
```

预演输出 `dry_run`，不复制文件。实际安装输出 `installed`；内容相同再次执行会输出 `already_installed`。若现有目录内容不同，会拒绝覆盖；确认升级后按后文备份方式执行。

项目级安装则用：

```
python scripts/install.py --host codex --scope project --project .
```

用户级与项目级选一种即可。项目级的好处是只影响当前仓库；用户级适用于多个研究目录。第一版 Codex ZIP 中包含相同的安装脚本和完整技能目录；它不是需要在 Python 中 `pip install` 的库。

3.3 新会话验收

在 Codex 中打开克隆的仓库作为工作区，新开会话并输入：

```
请使用 $mmhs-cryptographic-function-design。
先读取入口，列出它的 15 张能力卡；说明 DS 只允许 R_z=I，
以及同偏差校正阶和弹性阶的区别。
请给出实际读取路径，不要开始构造。
```

成功标准：找到入口、15 卡以及明确边界。若回答只有目录名称而不读取，明确要求打开 SKILL.md 及相应卡。模型能口头复述不等于程序已执行；下一章用实际数值验收。

4 第四步：先运行一个能核对的数值实例

4.1 MM：验证本机计算链路

以下命令在 CryptoBoolFunc 仓库根执行。它不调用收费模型 API，也不需要把原书上传到网络：

```
New-Item -ItemType Directory -Path output -Force | Out-Null
$tool = 'skills/mmhs-cryptographic-function-design/resources/math_tool.py'
python $tool mm --input examples/mm.json --output output/mm-result.json
$r = Get-Content output/mm-result.json -Raw | ConvertFrom-Json
$r.status
$r.metrics.nonlinearity
$r.metrics.balanced
$r.metrics.sac
```

应依次看到 ok、4、True 及 4 个 True。样例的输入为 k=3、选择器系数 3 与 4、常数项 1 与 0，得到 4 变量函数。不要将输入选择器长度 2 误解为 2 位选择器。工具采用整数 bit i 表示第 i 输入/输出坐标，原书其他排表方式须先换坐标再对比。

4.2 多输出：必须检查所有非零组合

```
python $tool ds --input examples/ds.json --output output/ds-result.json
$r = Get-Content output/ds-result.json -Raw | ConvertFrom-Json
```

```
$r.metrics.component_count
$r.metrics.min_nonlinearity
$r.metrics.balanced
```

应看到 7、240、True。因为 m=3，需要检查 7 个非零输出组合。`min_nonlinearity` 是最坏分量非线性度，`min_degree` 与 `max_degree` 分别报告两个次数口径。本版 DS 只实现经过复核的 $R_z = I$ 子族；指定自定义 R 会拒绝。一般可逆 R 分支已发现反例，不能因为示例跑通而恢复其一般结论。

4.3 校正器与阵列例

```
python $tool corrector --input examples/corrector.json --output output/corrector.json
python $tool d-array --input examples/d-array.json --output output/d-array.json
```

双码校正器应报告最小非线性度 448、最小次数 4、整体校正阶 4。它要求输入位统计独立且具有相同偏差；若模型条件改变，不能沿用相同保证。阵列例固定多项式 37，即 $x^5 + x^2 + 1$，合法行数应为 10；换为已核查的另一表示 47 时可为 11。容量要连同多项式、基和列选择一起记录。

5 第五步：让宿主按卡执行，而非只口头解释

在已打开正确工作区的宿主会话中提交：

```
请使用 mmhs-cryptographic-function-design 中的 mm-spectrum 卡。
输入取 examples/mm.json。
先列合法性条件，再运行资源程序，把输出写到 output/mm-session.json。
报告实际命令、退出码、完整表、Walsh 谱、非线性度与 SAC。
区分原书结论、本文重建和本次有限实例；不要扩大为一般最优。
```

成功标准有四项：读取正确卡；实际运行命令；输出文件存在且可解析；数值与前面手工验收一致。宿主如果需要权限批准，应核对具体命令和输出路径。API 失败、文件权限失败与数学条件不满足是三种不同失败，应分别记录。

再做一次边界验收：把 MM 输入的 phi 改成重复值，例如 [1,1]，使用另一个输入文件运行。工具应返回 error 且退出码 2，不能继续报告单射构造成功。无论哪个宿主，“明确拒绝非法输入”都是必要行为。

6 第六步：从新材料重新使用仓颉蒸馏

如果只是使用本版 15 项方法，不必重新蒸馏。若要扩展到新论文、另一份书稿或新章节，将合法持有的 PDF 留在本机 `local-source` 中，向宿主给出真实路径和目标。例如：

```
请使用 cangjie-skill 处理 local-source/my-paper.pdf。
目标：服务新型布尔函数/多输出函数的设计。
先建立章节/构造/定理依赖图，重点拆构造思想、参数限制和证明步骤。
每条候选定位到页码与公式；一般证明缺口、开放问题及数表冲突要隔离。
保存日志，先交整书/整文骨架，不凭记忆补造原文。
```

仓颉大致经历：整书理解、五类提取、三重验证、独立入口晋级、RIA 能力卡、关联与词典、压力测试、确定性编译 [1]。用户在阶段门审阅的是具体结果，不是用确认代替来源验证。一次只做一个内容试点；中断后读取状态文件续跑，不必重复已经核查的材料。

要求每项构造输出四类证书：**输入契约**（域/基/位序/维数）、**生成结果**（真值表或确定性生成器）、**证明链**（局部谱到整体指标的每一步）、**计算范围**（哪些参数实测、哪些只属一般推导）。未被完整支持的内容进入待核库，不能为了增加卡数量而包装成 active 方法。

7 读懂第一版的验证报告

本版正常任务 15/15 完成、边界语义 15/15，路由 18/18；原严格字段断言 28/30。两处 error 与 not_applicable 状态名差异仍计严格不匹配，不能把“数学上正确拒绝”写成“所有字段断言通过”。单次对照也不是统计显著提升的证明。

第一版编号 1.0.0 是可追溯快照名，保留研究试用属性。尚未解决一般 DC 存在性、完整 HML/FAI 求解器、混合阵列一般精确界，也不保证所有密码攻击指标。源书并未随公共仓库分发，出处和来源哈希保留用于审计。

8 把安装与研究过程写成日志

建议每次实验保存如下记录，而不是只保存模型对话：

```
version: 1.0.0
host: Codex or DeepSeek Harness
input: examples/mm.json
skill: mm-spectrum
domain_and_basis: explicit
command_and_exit_code: recorded
observed_metrics: from output JSON
proof_scope: instance / family / lower bound
unverified_items: explicit list
```

日志不能包含 API key。输出文件内容与模型摘要冲突时，先复算程序和输入；不要为了让两者一致而手工改指标。修改算法后至少重跑相关正常例和边界例，再生成新版本。

9 Codex 专用排错与项目使用

现象	操作与成功标志
找不到技能	检查用户/项目扫描根，确保 name 与目录格式正确；重开会话，要求报告实际文件路径。
同名技能出现两次	查找用户级、项目级和旧兼容目录；只保留本次选定来源，先备份再移动。
不能运行 Python	在宿主终端核查解释器路径；使用已验证的 Python 绝对路径。系统终端可运行不代表宿主 PATH 相同。
只能看到文件但不能运行	检查当前沙箱/权限策略；先读与预演，依据宿主批准流程执行必要命令，不通过关闭所有限制解决。
模型说计算完成却无文件	要求保存输入 JSON、输出 JSON、命令、退出码和所有非零分量指标；人工打开输出核对。

后续新论文或新函数候选，先用仓颉做来源与任务拆解，再用本项目做限定构造和验证。两者都要留日志；用同一个词“验证通过”时，务必说明是来源、公式推导、脚本实例还是在线宿主任务。

10 升级、备份与跨平台操作

10.1 升级不覆盖旧研究

先查看现有安装路径、版本和差异，再运行：

```
python scripts/install.py --host codex --scope user --upgrade --dry-run
python scripts/install.py --host codex --scope user --upgrade
```

DeepSeek 版本把 host 值改为 `deepseek`。若使用项目级，保留原来的 scope 和 project 参数。旧目录会存入扫描根的 `.backups/时间戳/技能名`，安装器报告具体位置；不自动删除历史研究目录。需要回退时先停止宿主，再将当前目录另存，恢复指定备份，重新验收。不要同时装两份同名旧版和新版来“试试”。

10.2 macOS/Linux 差异

将 `python` 统一改为实际可用的 `python3`，安装和测试命令其他参数不变。下载仓颉可用：

```
mkdir -p "$HOME/CryptoBoolFunc-Lab"
cd "$HOME/CryptoBoolFunc-Lab"
base=https://github.com/kangarooking/cangjie-skill/releases/download/v2.5.0
curl -fL "$base/cangjie-skill-2.5.0.zip" -o cangjie.zip
curl -fL "$base/cangjie-skill-2.5.0.zip.sha256" -o cangjie.sha256
shasum -a 256 cangjie.zip
```

```
cat cangjie.sha256
unzip cangjie.zip -d upstream
python3 -m pip install pyyaml
python3 upstream/cangjie-skill/scripts/cangjie.py doctor
```

逐字比较两个哈希后才继续。Linux 可用 `sha256sum`。将完整 `upstream/cangjie-skill` 复制到所选宿主扫描根，或克隆本项目后用安装器的 `--cangjie-source` 参数一起安装。安装器通过 Python 解析用户目录，因此无需手工翻译 Windows 路径。

11 重新编译本教程 PDF

安装带中文支持的 TeX Live 或 MiKTeX。使用 XeLaTeX，而不是 pdfLaTeX。本教程采用 Fandol 中文字体与 Latin Modern Mono，避免依赖私人电脑字体。

```
python scripts/build_docs.py
```

此脚本对两份 TeX 各编译两遍，临时文件留在忽略的.build 目录，PDF 复制到 docs。手动编译则先进入 docs：

```
xelatex -interaction=nonstopmode -halt-on-error tutorial-codex-v1.tex
xelatex -interaction=nonstopmode -halt-on-error tutorial-codex-v1.tex
```

另一份把文件名改为 `tutorial-deepseek-v1.tex`。成功不只看“生成 PDF”：还要查看全部页的标题、中文、代码换行、链接和页码。日志中的 Overfull、Missing character 和 LaTeX Error 需要处理；不能把异常日志隐藏后继续发布。

12 以后怎样按固定流程上传 GitHub

本仓库没有一份可以凭空套用的“统一最新强制上传规范”；项目采用核查日 GitHub 官方文档和自身规则建立流程 [7, 8]。后续按 `RELEASE_PROCESS.md` 执行：

1. 读取远端默认分支、最新 SHA 与保护规则，检查本地变更；先确定本次授权范围。
2. 更新版本、说明、数学边界及 Codex 贡献记录；只提交公开清单内的文件。
3. 运行测试，重编 TeX/PDF 并视觉验收；生成两个宿主 ZIP、SHA256SUMS 与 manifest。
4. 建立 `codex/v 版本-主题`分支并提交；上传后创建 PR，说明变化、验证与未测项。
5. 在授权和仓库规则允许时合并；有失败检查或审批要求则保持 PR 并说明条件，不强推。
6. 再读远端 main/合并提交，核对 PDF、ZIP 和提交 SHA，保存上传日志。

原始 PDF、全文抽取、个人绝对路径、密钥和编译缓存不进入公共提交。仓库的 `releases/v1.0.0` 是本项目版本资产目录；只有确实创建了 GitHub Release 和标签，才称创建了该平台的 Release，二者不能混同。

13 完成检查表

1. 仓颉完整目录可读，doctor 为 PASS，版本和下载哈希有记录。
2. 当前宿主在正确工作区发现本项目入口，能读取 15 张卡和对应资源。
3. MM 输出 N=4；DS 示例 N=240 且 7 分量；双码校正器 N=448；D 阵列固定表示得到 10。
4. 非法输入被拒绝，DS 一般 R 分支与其他待核项没有被重新启用。
5. 安装器、版本、PDF/TeX、SHA256 与贡献者说明均可查；个人实验输出已另存。

贡献与引用

本项目由 wernerchang 发起和维护。Codex（OpenAI）作为 AI 协作贡献者，协助研究整理、程序、验证、教程和发布准备；具体记录见 `CONTRIBUTORS.md`。仓颉上游和《密码函数》作者分别署名。AI 协作不替代维护者对学术结论的判断。

参考文献

[1] kangarooking. Cangjie Skill：官方仓库及v2.5.0 刷新附件。核查 2026-11-04。

[2] OpenAI. Build skills。本地目录、完整技能与调用方式。

[3] OpenAI. Codex quickstart。

[4] DeepSeek AI. DeepSeek Harness 官方 README，开发预览与启动方式。

[5] DeepSeek AI. Filesystem skill provider，扫描根及目录格式。

[6] DeepSeek AI. Web UI 指南，模型与工作区。

[7] GitHub Docs. Creating a pull request。

[8] GitHub Docs. About releases。